```
subjectKeyIdentifier = hash
Source : docs.vapor.build
```

You can dispatch tasks in typical Laravel apps that will be run when the HTTP response is given to the browser:

```
Route::get('/', function () {
  dispatch(function () {
   Mail::to(abc@example.com')->send(new WelcomeMessage);
  })->afterResponse();
  return view('home');
});
```

When utilizing Vapor, we always recommend dispatching jobs to your queue workers. Because Vapor cannot run a task after receiving a response from the browser, attempting to do so will make your application look slower to the end user.

Domains

When you use Vapor, each environment is assigned a unique vanity domain that you may use to access the environment once it has been deployed. However, bear in mind that vanity domains are exclusively for development purposes, and Vapor automatically inserts *no-index* headers to replies from these URLs to prevent search engines from indexing them.

Furthermore, even while an application is in maintenance mode, these vanity domains will continue to work, allowing you to test your environment before disabling maintenance mode and restoring general access.

While vanity domains might be beneficial during the development phase, they should not be used in a production setting or shared with the public. Instead, it is recommended to utilize your own custom code.

You may add a domain that you own to Vapor via the domain screen or the domain CLI command. Even if you intend to serve your application via a subdomain, always add the root domain when adding a domain to Vapor:

```
vapor domain example.com
```

To inspect the specifics of your Vapor domains using the Vapor UI or the **`domain:list`** CLI command:

```
vapor domain:list
```